

AR Report - Augmented Rap Collection

Francesco Frattini - 5721355

1 Introduction

The **Augmented Rap Collection** is an Augmented Reality mobile application that transforms a curated selection of **Italian rap album covers** into **interactive AR markers**.

By pointing the device camera at one of five physical album covers (Marracash's *Persona*, Tedua's *Orange County California*, Ernia's *Per Soldi e per Amore*, Salmo's *Hellvisback*, and Lazza's *Re Mida*), the application **recognizes the cover** and **overlays a floating information card** displaying details about the album: release year, genre, key tracks, and a short fun fact.

The application is designed around a non-linear, exploratory interaction model, where each album cover is an independent trigger. Multiple cards can appear simultaneously if several covers are within the camera's field of view, reflecting a natural browsing behavior rather than a scripted flow.

The project was developed and tested both in Unity's XR Simulation environment (using a custom virtual living room designed to reflect the project's musical theme, featuring a record player, a piano, and speakers) and on a physical Android device (Google Pixel 8a), validating the application in real-world AR conditions.

2 Libraries, SDKs and Technical Stack

The application was built using **Unity 6.4** (6000.4.1f1) with the Universal Render Pipeline (**URP**), chosen for its native compatibility with AR Foundation and its optimized rendering performance across both desktop simulation and mobile platforms.

AR Foundation (Unity's cross-platform AR framework) was selected as the core AR library, providing a unified API that abstracts the underlying platform-specific implementations. This architectural choice allows the exact same codebase to execute seamlessly in both the Unity Editor (via XR Simulation) and on the physical Android device (via ARCore), without requiring any changes to the core application logic.

The following specific packages were used:

- **AR Foundation:** Core framework providing the essential `ARSession`, `XROrigin`, and `ARTrackedImageManager` components.
- **ARCore XR Plugin:** Platform-specific provider for Android, enabling real-world AR on the Google Pixel 8a via Google's native ARCore services.
- **XR Simulation:** Unity's built-in simulation environment, allowing full AR testing directly inside the Editor without a physical device.
- **TextMeshPro:** Used for all UI text rendering inside the information cards.

The **image tracking** feature (`ARTrackedImageManager`) was chosen as the primary detection method over alternatives such as plane detection or object tracking. Album covers are visually rich, high-contrast, and flat surfaces: ideal targets for marker-based image tracking, which provides stable and precise pose estimation compared to markerless approaches.

A **custom XR Simulation environment** (`LivingRoom_RapCollection`) was built on top of Unity’s predefined sample environments, extended with thematic 3D assets to reflect the musical context of the application. This environment served as the primary testing ground during development. All five album covers were integrated as `SimulatedTrackedImage` components, allowing full end-to-end validation of the tracking and UI pipeline without requiring physical hardware at every iteration.

3 Code Architecture and Design Choices

The application follows a clean separation of concerns between the data layer, user interface, and tracking logic, organized into four dedicated scripts and a custom `ScriptableObject` architecture.

3.1 Data Layer: `AlbumData.cs`

Album information is managed using Unity’s `ScriptableObject` system. Each of the five albums is represented by an independent `.asset` file containing: the reference image name (used for matching), artist name, album title, release year, genre, key tracks, and a short fun fact. This architectural approach decouples the content entirely from the application logic: adding, removing, or modifying an album requires no code changes, only editing the corresponding asset fields within the Unity Inspector. The `referenceImageName` string acts as the explicit unique identifier linking an `AlbumData` asset to its corresponding entry in the AR Reference Image Library, ensuring a robust and easily verifiable matching process.

3.2 Tracking Logic: `AlbumTrackingManager.cs`

This core component, attached to the `XROrigin` `GameObject`, listens directly to the `trackablesChanged` event exposed by the `ARTrackedImageManager`. It processes three fundamental lifecycle events:

- **Added / Updated:** Processed through a unified `ProcessTrackedImage()` method, which evaluates two conditions before rendering a card. First, the image’s tracking state must be actively `Tracking`. Second, the Euclidean distance between the camera and the detected target must be within a configurable threshold (`maxTrackingDistance`, optimized at 2.5 m). If both conditions are met and no card exists yet for that specific trackable, the prefab is instantiated as a local child of the tracked image’s transform, automatically inheriting its spatial position and orientation updates. If a card already exists, its visibility is toggled via `SetActive()`, preventing expensive and repetitive instantiation/destruction cycles at the boundary of the detection range.
- **Removed:** The associated card UI is destroyed and its reference is purged from the internal collection.

A `Dictionary<TrackableId, GameObject>` maps each active trackable identifier to its corresponding spawned card instance. This structure guarantees that each unique album cover corresponds to exactly one UI instance, regardless of how frequently its tracking state fluctuates.

A deliberate design choice was made regarding the handling of the `Limited` tracking state. When the user approaches a cover closely, the underlying computer vision algorithm temporarily loses confidence due to perspective skew and edge clipping, transitioning the state to `Limited`. Rather than allowing the UI to flicker or disappear abruptly, the card state is intentionally left unchanged: neither explicitly shown nor hidden, preserving

whatever visibility it had prior to the state transition, preserving UI stability and enhancing the overall user experience.

3.3 UI Layer: AlbumCardUI.cs

A single, reusable `AlbumCard` prefab is utilized across all five albums. It consists of a World Space Canvas hosting a semi-transparent background panel and four `TextMeshPro` text fields: artist and title, year and genre, key tracks, and the fun fact. Upon instantiation, the `AlbumCardUI.SetData()` method dynamically populates these fields using the injected `AlbumData` asset. This approach eliminates the overhead of maintaining five separate graphical prefabs and enforces complete visual and layout consistency by construction.

3.4 Billboard Behavior: BillboardUI.cs

To guarantee optimal legibility from any viewing angle, each floating card continuously reorients itself toward the main camera. The rotation calculation is constrained exclusively to the horizontal axis (the Y-component of the target direction vector is zeroed before computing the final `Quaternion.LookRotation`), preventing unnatural vertical tilting when the user views the card from significantly low or high angles. This logic is executed inside the `LateUpdate()` lifecycle method to ensure that the rotation occurs immediately after Unity has finalized all camera and tracking position updates for that frame.

3.5 AR Reference Image Library

The five target covers are registered within a Unity `XRReferenceImageLibrary` asset (`AlbumImageLibrary`), with physical dimensions defined as $0.5\text{ m} \times 0.5\text{ m}$ to match the `SimulatedTrackedImage` dimensions in the desktop environment. The `referenceImageName` of each sub-entry maps exactly to the corresponding field of the `AlbumData` asset, establishing an explicit lookup chain:

Detected Image $\longrightarrow$ Reference Name $\longrightarrow$ AlbumData Asset $\longrightarrow$ Card UI Content

4 Limitations and Possible Future Developments

While the application successfully demonstrates a complete marker-based AR pipeline, a number of limitations inherent to the chosen approach are worth acknowledging.

Marker visual quality dependency. The reliability of image tracking is directly tied to the visual richness of the target cover. Album artworks featuring strong contrast, diverse color gradients, and complex graphical detail, such as those selected for this project, provide stable and consistent tracking. Covers that are predominantly monochromatic, low-contrast, or physically damaged (creased, faded, or partially occluded) may result in degraded detection performance or increased latency in recognition. This is an intrinsic constraint of feature-based image tracking algorithms, independent of the application logic.

Interference between spatially proximate markers. When two or more album covers are placed in close physical proximity, the tracking system may occasionally misidentify one marker as another, particularly at intermediate viewing distances or oblique angles. This behavior is a known challenge in multi-marker image tracking scenarios and becomes more pronounced as the angular separation between targets decreases from the camera's perspective.

Static content and potential API integration. In its current form, all album metadata is authored manually and embedded as static `ScriptableObject` assets. A natural evolution of the project would involve replacing or augmenting this static layer with real-time data fetched from external sources, such as music analytics

platforms, to provide dynamically updated information. The existing `AlbumData` architecture is deliberately designed to support this extension: replacing the static fields with API-populated values would require no structural changes to the tracking or UI layers.

5 Repository, Setup and Testing

5.1 Source Code Repository

The complete Unity project is publicly available at the following GitHub repository:

<https://github.com/francii0003/AugmentedRapCollection>

The repository includes all project assets, scripts, `ScriptableObject` data files, the custom XR Simulation environment (`LivingRoomRapCollection`), and the AR Reference Image Library. Build artifacts and temporary Unity cache folders are excluded via a standard Unity `.gitignore`.

5.2 Running the Project

The project can be run in two modes.

XR Simulation (Editor). To run the application directly inside the Unity Editor without any physical device:

1. Open the project in **Unity 6.4** (6000.4.1f1) or later.
2. Ensure the following packages are installed via the Package Manager: **AR Foundation**, **ARCore XR Plugin**, **XR Simulation**.
3. Open the scene `Assets/Scenes/MainScene.unity`.
4. Verify that `LivingRoomRapCollection` is selected as the active environment in `Window > XR > AR Foundation > XR Environment`.
5. Press **Play**. Navigate the simulation using right-click + WASD to move the virtual camera toward the album covers and trigger the information cards.

Android Device. To build and run the application on a physical Android device (ARCore-compatible, Android 8.0 or later):

1. Switch the build platform to **Android** via `File > Build Profiles`.
2. Enable **ARCore** under `Edit > Project Settings > XR Plug-in Management > Android`.
3. Connect an ARCore-compatible Android device via USB with USB Debugging enabled.
4. Select `Assets/Scenes/MainScene.unity` in the Scenes in Build list.
5. Press **Build and Run**. Grant camera permissions when prompted on the device.
6. Point the device camera at one of the five physical album covers to trigger the corresponding information card.

5.3 Testing and Demonstration Video

The application was tested against all five target album covers, both in **XR Simulation** and on a physical **Google Pixel 8a** running Android with ARCore. All five markers were successfully recognized, with information cards appearing correctly upon detection and being dismissed upon the camera moving out of range or the tracking state dropping below the configured threshold. Two demonstration videos showing the running application in both simulation and real-world conditions are available at:

<https://github.com/francii0003/AugmentedRapCollection/blob/main/VideoSimulation.mp4>

<https://github.com/francii0003/AugmentedRapCollection/blob/main/VideoAndroid.mp4>